

SLR grammar (" is s):

- (0) PROGRAMA ->
- (1) DECLARACAO
- (2) PROGRAMA ->
- (3) PROGRAMA
- DECLARACAO
- DECLARACAO -> definir
- variavel como tipo string
- DECLARACAO ->
- TIPO_TOKEN variavel =
- new CLASSE_TOKEN

FIRST / FOLLOW table		
Nonterminal	FIRST	FOLLOW
PROGRAMA	{definir, TIPO_TOKEN}	{\$, definir, TIPO_TOKEN}
DECLARACAO	{definir, TIPO_TOKEN}	{\$, definir, TIPO_TOKEN}

>>

SLR closure table			
Goto	Kernel	State	Closure
	{PROGRAMA -> .DECLARACAO}	0	{PROGRAMA -> .DECLARACAO; DECLARACAO -> .definir variavel como tipo string; DECLARACAO -> .TIPO_TOKEN variavel = new CLASSE_TOKEN}
goto(0, DECLARACAO)	{PROGRAMA -> DECLARACAO.}	1	{PROGRAMA -> DECLARACAO.}
goto(0, definir)	{DECLARACAO -> definirvariavel como tipo string}	2	{DECLARACAO -> definirvariavel como tipo string}
goto(0, TIPO_TOKEN)	{DECLARACAO -> TIPO_TOKEN.variavel = new CLASSE_TOKEN}	3	{DECLARACAO -> TIPO_TOKEN.variavel = new CLASSE_TOKEN}
goto(2, variavel)	{DECLARACAO -> definir variavel.como tipo string}	4	{DECLARACAO -> definir variavel.como tipo string}
goto(3, variavel)	{DECLARACAO -> TIPO_TOKEN variavel.= new CLASSE_TOKEN}	5	{DECLARACAO -> TIPO_TOKEN variavel.= new CLASSE_TOKEN}
goto(4, como)	{DECLARACAO -> definir variavel como.tipo string}	6	{DECLARACAO -> definir variavel como.tipo string}
goto(5, =)	{DECLARACAO -> TIPO_TOKEN variavel =.new CLASSE_TOKEN}	7	{DECLARACAO -> TIPO_TOKEN variavel =.new CLASSE_TOKEN}
goto(6, tipo)	{DECLARACAO -> definir variavel como tipo.string}	8	{DECLARACAO -> definir variavel como tipo.string}
goto(7, new)	{DECLARACAO -> TIPO_TOKEN variavel = new.CLASSE_TOKEN}	9	{DECLARACAO -> TIPO_TOKEN variavel = new.CLASSE_TOKEN}
goto(8, string)	{DECLARACAO -> definir variavel como tipo string.}	10	{DECLARACAO -> definir variavel como tipo string.}
goto(9, CLASSE_TOKEN)	{DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN.}	11	{DECLARACAO -> TIPO_TOKEN variavel = new CLASSE_TOKEN.}

LR table											
State	ACTION								GOTO		
	definir	variavel	como	tipo	string	TIPO_TOKEN	=	new	CLASSE_TOKEN	\$	PROGRAMA
0	s2					s3					1
1	acc					acc				acc	
2		s4									
3		s5									
4			s6								
5							s7				
6				s8							
7								s9			
8					s10						
9									s11		
10	r2					r2				r2	
11	r3					r3				r3	

Input (tokens):

Maximum number of steps:

PARSE

Trace					Tree
Step	Stack	Input	Action		
1	0	id + id * id \$			